HDOC Day-67

Finding the GCD of Two Positive Integers Using Recursion and Euclid's Algorithm in C

Aim

To write a C program using recursion, user-defined functions, conditional statements, and Euclid's algorithm to find the Greatest Common Divisor (GCD) of two positive integers. The program reads two positive integers, calls gcd(), displays their GCD, and reports invalid non-positive input.

Algorithm

1. Start the program and declare a, b, and result.
2. Read two positive integers from the user.
3. If either integer is zero or negative, display an error and stop.
4. Define a recursive function gcd(int a, int b).
5. If b equals zero, return a (base case).
6. Otherwise, recursively call gcd(b, a % b).
7. Continue until the second argument becomes zero.
8. Return the result to main(), display the GCD, and stop.

C Program

```c
#include <stdio.h>

/* Function declaration */
int gcd(int a, int b);

int main()
{
    int a, b, result;

    printf("Enter first positive integer: ");
    scanf("%d", &a);

    printf("Enter second positive integer: ");
    scanf("%d", &b);

    if (a <= 0 || b <= 0)
    {
        printf("Invalid input! Enter positive integers only.\n");
        return 0;
    }

    result = gcd(a, b);

    printf("\nGCD of %d and %d is: %d\n", a, b, result);
    return 0;
}

/* Recursive function using Euclid's Algorithm */
int gcd(int a, int b)
{
    if (b == 0)
        return a;

    return gcd(b, a % b);
}
```

Explanation

The Greatest Common Divisor (GCD) is the largest positive integer that divides both given integers without leaving a remainder. For example, the common factors of 12 and 18 are 1, 2, 3, and 6; therefore GCD(12, 18) = 6.

Euclid's algorithm uses the identity **GCD(a, b) = GCD(b, a % b)**. The recursion ends when b becomes zero, at which point the function returns a.

Worked Example: gcd(48, 18)

Step	Function call	Calculation
1	gcd(48, 18)	48 % 18 = 12
2	gcd(18, 12)	18 % 12 = 6
3	gcd(12, 6)	12 % 6 = 0
4	gcd(6, 0)	Return 6

The calls progress from gcd(48, 18) to gcd(6, 0), where the base case returns 6. Each earlier recursive call then returns the same result.

How Recursion Works

```
int gcd(int a, int b)
{
    if (b == 0)        /* Base case */
        return a;
    return gcd(b, a % b); /* Recursive case */
}
```

The algorithm has logarithmic time complexity and logarithmic auxiliary space complexity because each recursive call uses a stack frame.

Test Case Table

Test	First integer	Second integer	Expected result
1	48	18	6
2	12	18	6
3	24	36	12
4	17	13	1
5	100	25	25
6	7	7	7
7	81	27	27
8	56	98	14
9	0	15	Invalid input
10	-12	18	Invalid input

Sample Outputs

Test Case 1: Two Positive Integers

```
Enter first positive integer: 48
Enter second positive integer: 18

GCD of 48 and 18 is: 6
```

Test Case 2: Coprime Integers

```
Enter first positive integer: 17
Enter second positive integer: 13

GCD of 17 and 13 is: 1
```

Test Case 3: One Number Divides the Other

```
Enter first positive integer: 100
Enter second positive integer: 25

GCD of 100 and 25 is: 25
```

Test Case 4: Equal Numbers

```
Enter first positive integer: 7
Enter second positive integer: 7

GCD of 7 and 7 is: 7
```

Test Case 5: Invalid Input

```
Enter first positive integer: -12
Enter second positive integer: 18

Invalid input! Enter positive integers only.
```

Result

The C program was designed using recursion, user-defined functions, conditional statements, and Euclid's algorithm. The gcd() function repeatedly replaces its arguments with the second integer and the remainder until the second argument becomes zero. The program handles positive integers, coprime and equal numbers, and invalid non-positive inputs without iterative loops.